Current Recursion and Tensor-Network Execution in pyAmpliCol

Purpose. This note summarizes the matrix-element generation strategy used by AmpliCol and the corresponding pyAmpliCol implementation. The common idea is to avoid explicit Feynman-diagram enumeration. Instead, matrix elements are assembled from a directed acyclic graph (DAG) of off-shell currents. AmpliCol emits Fortran code that numerically sweeps this current table. pyAmpliCol builds the same logical DAG in Python, lowers bounded pieces to tensor-network expressions handled by Spenso, compiles the resulting parametric scalar/vector expressions with Symbolica evaluators, and can now execute the staged sweep from Rust through the PyO3 runtime `rusticol`.

Current status. The generic LC/NLC/full-colour staged-DAG implementation is the production pyAmpliCol workflow. Process generation writes schema-v2 artifacts loaded by `rusticol`; model parameters are runtime inputs and may be overridden from TOML. The default benchmark route is Rusticol with SymJIT stage evaluators and stage-local inputs. Selected-flow JIT artifacts measure stage chunks around base 128 and run at batch 128; all-flow artifacts use uniform chunk 8192 and batch 64. Both use ten Horner iterations, Symbolica's default CPE choice, and enlarged Horner/common-pair limits. The production matrix JIT row uses SymJIT O1 for generation-speed comparisons, with a starred selected-flow O3 result only where repeated ten-second timings establish a material runtime gain; dedicated Z-family scans also show O3/ASM/C++ variants. Selected JIT generation benchmarks effective chunk sizes 64, 96, 128, 192, 256, and unchunked for each stage, retaining a non-default candidate only above a five-percent measured gain. On AArch64, candidate scores count one shared native input pack and sum only evaluator and output-unpack work across chunks, matching Rusticol's fused stage call. Large all-flow and backend-comparison artifacts stay uniformly chunked. The result matrices are generated as separate TEX inputs, and the benchmark writers refresh `pyAmpliCol.pdf` after each completed cell. Generic current and colour-sector generation caps are disabled by default; long benchmark refreshes rely on the 30 GB watchdog rather than truncated colour plans. On macOS the watchdog enforces the larger of process-tree RSS and compressed physical footprint.

Once runnable stage evaluators are serialized, the saved schema compacts full generation-only interaction records to stage interaction IDs and vertex-kind counts. The full records remain available in `runtime_schema_diagnostics.pickle.gz`; plan-only manifests stay detailed, and Rusticol remains backward-compatible with earlier artifacts. On measured all-flow artifacts this removes about 80 percent of manifest bytes without changing any evaluator, runtime schedule, or matrix-element value.

1 The AmpliCol Current Recursion

For a fixed leading-color ordering, a current is labelled by the external subset it contains, the particle type carried by the off-shell line, and any spin/chirality information. Schematically, if I is a set or ordered interval of external colored legs, a current is computed as

$$\mathcal{J}^{\alpha}_{t,\chi}(I) = P^{\alpha}{}_{\beta}(t, p_I) \sum_{\substack{I=A\cup B\\ A\cap B=\emptyset}} \mathcal{V}^{\beta}{}_{\gamma\delta}(t_A, t_B \to t)\, \mathcal{J}^{\gamma}_{t_A,\chi_A}(A)\, \mathcal{J}^{\delta}_{t_B,\chi_B}(B). \tag{1}$$

Here t is the current type (gluon, quark, auxiliary tensor, etc.), χ is the Weyl chirality where relevant, $p_I = \sum_{i\in I} p_i$, P is the propagator, and $\mathcal{V}$ is the appropriate Feynman-rule tensor. The four-gluon vertex is treated through the auxiliary antisymmetric tensor route, so it is still represented by cubic current combinations.

AmpliCol's optimized mode does not loop over full helicity configurations as independent computations. It weaves helicity information into the current table itself. Each external helicity state is a source current with a unique bit in an `ext_cur`-style bitset. An internal current is identified by

$$K(\mathcal{J}) = \big(t, \chi, \mathrm{bin}(I), \mathrm{ext\_cur}\big), \tag{2}$$

where $\mathrm{bin}(I)$ records the external subset and ext_cur is the union of the source-current bits feeding the current. When a new vertex contribution produces the same key as an existing current, AmpliCol appends that vertex to the existing current rather than creating a duplicate node. After all currents are constructed, `curr2amp` records the two endpoint currents whose contraction gives each retained helicity amplitude.

The generated Fortran library has the same layered structure:

```
subroutine evaluate_amp(p, amps)
  complex(kind=8) :: val_c(1:6, n_cur)
  complex(kind=8) :: int_c(1:6, n_vert)

  call compute_external_currents(pp, val_c)
  do size = 2, n_external_minus_one
    call compute_vertices_size(pp, val_c, int_c)
    call compute_currents_size(pp, val_c, int_c)
  end do
  call compute_amps(amps, val_c)
end subroutine
```

The arrays `val_c` and `int_c` are the numerical current and interaction tables. This is the performance-critical idea that pyAmpliCol keeps: one topological sweep produces all retained amplitudes, and shared parent currents are evaluated once.

2 The pyAmpliCol DAG

pyAmpliCol first builds an explicit Python representation of the same current table. In the generic compiler, a current is identified by a model-driven physics index rather than by a process-family name. The identity used for deduplication is

$$K_{\text{pyAmpliCol}} = \big(\text{particle\_id}, L, H, \chi, S, F, Q, C, P, A\big), \tag{3}$$

where L is the sorted tuple of external labels, H is the AmpliCol-style helicity-source ancestry bitset, χ is chirality, S is any spin-state tag needed by the source or propagator, F is flavour flow, Q is charge flow, C is the leading-color state, P is the momentum mask, and A is an optional auxiliary kind such as the antisymmetric tensor used for the four-gluon vertex. The implementation also carries `ordered_external_labels`. That field is leading-color ordering metadata used by the color engine and amplitude grouping, but it is deliberately not part of equality or hashing. Deduplication is therefore exactly equality of Eq. (3); no rule is allowed to ask whether the process is "Z plus jets" or any other family.

field	role in the current identity
`particle_id`	Model/UFO particle id, e.g. 21, 23, 1, -1, or an auxiliary id.
`external_labels` and `external_mask`	Which external legs are contained in the current. The mask is the bit representation of the labels.
`helicity_ancestry`	Bitwise union of the source-current helicity bits feeding this current. This is how helicity configurations are woven into one shared DAG.
`chirality` and `spin_state`	Local spin information, especially for Weyl fermions and massive-vector sources.
`flavour_flow` and `charge_flow`	Quantum-number flow checked locally by model vertices, e.g. $u \to dW^+$ or $e^+e^- \to \gamma/Z$.
`color_state`	Colour-ordered open-line or trace state used to build the amplitude vector. LC, NLC and full colour differ in the final sparse colour contraction over this vector.
`momentum_mask`	Which all-outgoing momenta are summed for propagators and momentum-dependent vertices.
`auxiliary_kind`	Optional tag distinguishing auxiliary tensor currents from ordinary particles.

Source currents are built from external leg metadata. Internal currents are then generated by increasing current size, with all interactions that feed the same current index attached to one node.

```
for external helicity source s:
    add_current(key=(pdg_s, labels_s, chirality_s, bit_s), is_source=True)

for current_size in increasing_sizes:
    for compatible split A | B:
        result_key = combine_key(key[A], key[B], vertex_kind)
        result = add_or_reuse_current(result_key)
        add_interaction(left=A, right=B, result=result, vertex=vertex_kind)

for retained helicity ancestry:
    add_amplitude(left=current_with_Z_and_gluons, right=anti_current)

prune_dead_currents_after_helicity_filter()
```

At runtime, pyAmpliCol mirrors the Fortran table layout with contiguous arrays:

$$C_{s,i,c} \equiv \texttt{current\_rows}[s,i,c], \qquad T_{s,v,c} \equiv \texttt{interaction\_rows}[s,v,c],$$

where s is the phase-space sample in the current batch, i is a current id, v is an interaction id, and c is a component index. External wavefunctions fill source-current rows once per sample. Bounded evaluator blocks then fill interactions and derived currents stage by stage:

```
fill_source_currents(current_rows, momenta, helicity_sources)

for stage in current_size_layers:
    state = ParamBuilder.pack(current_rows, interaction_rows, momenta)
    interaction_rows[:, stage.vertices] = vertex_eval[stage](state)
    current_rows[:, stage.currents]     = current_eval[stage](state)

amps = amplitude_eval(ParamBuilder.pack(current_rows, momenta))
me2  = normalization * sum(weight[h] * abs(amps[h])**2 for h in retained_h)
```

This is why the evaluator-only benchmark is meaningful: the expensive recursion is inside compiled evaluator calls, while Python only performs source wavefunction construction, parameter packing, and table transfers.

3 Spenso Tensor Networks and `hep_lib`

Spenso is used to express Lorentz, spinor, auxiliary-tensor, and eventually color contractions in a representation-aware way. A tensor-network expression contains named tensors with typed slots:

$$\mathcal{V}_{\mu\nu}{}^{\rho}(p_A,p_B)\,A^{\mu}\,B^{\nu}, \qquad \text{or in code: } \texttt{V(m("a"),m("b"),m("out")) * A(m("a")) * B(m("b"))}.$$

The expression is not expanded. It is a compact product of tensor factors with shared representation labels. The library `hep_lib` is the dictionary that tells Spenso what each tensor name means: dense model tensors, metric tensors, momentum-dependent propagators, and parametric rank-one tensors representing runtime inputs.

```
library = model.build_tensor_library()      # returns TensorLibrary.hep_lib_atom()
builder = ParamBuilder()
mink = Representation.mink(4)

builder.register_rank1_tensor(
    library,
    tensor_name="current_A",
    representation=mink,
    head=("stage", "A"),
    length=4,
    role="block_current",
)
builder.register_rank1_tensor(
    library,
    tensor_name="p_A",
    representation=mink,
    head=("stage", "pA"),
    length=4,
    role="block_momentum",
    real_valued=True,
)

raw = V3g(mink("a"), mink("b"), mink("out"), p_A="p_A", p_B="p_B") \
    * TensorName("current_A")(mink("a")).to_expression() \
    * TensorName("current_B")(mink("b")).to_expression()

network = TensorNetwork(raw, library)
network.execute(library=library)
out_tensor = network.result_tensor(library)
evaluator = out_tensor.evaluator(params=builder.parameter_symbols())
```

The important point is that `hep_lib` stays fixed during the execution. New graph currents are not registered as permanent physics tensors. Instead, the current values are runtime parameters for a bounded block. Spenso executes the factorized tensor network, contracts the representation indices, and returns a parametric output tensor. Symbolica then compiles that parametric tensor to an evaluator. For high multiplicity, pyAmpliCol interleaves this process: after a current or current-size layer is contracted, its numerical output is written back into the current table and becomes an input parameter for later blocks. This avoids one giant scalar expression and keeps the DAG close to AmpliCol's execution model.

The current NLC/full-colour implementation keeps the kinematic DAG colour-ordered and applies a sparse colour matrix to the amplitude vector. For open-line sectors, including arbitrary many quark lines with gluons distributed on the lines, pyAmpliCol contracts the product of left colour strings with the conjugated right strings into closed fundamental traces. Those small colour trace expressions are simplified separately from the kinematic DAG; NLC is obtained by keeping the leading and first $1/N_c^2$-suppressed powers, while full colour keeps the complete sparse metric. If a later implementation puts explicit colour tensors inside a Spenso block, colour simplification still belongs before Spenso execution:

$$\texttt{raw} \leftarrow \texttt{idenso.simplify\_color(raw)}, \qquad \text{then build } \texttt{TensorNetwork(raw, hep\_lib)}.$$

For the present NLC/full-colour milestone, colour algebra is confined to the small colour-basis overlap problem and the Rusticol final contraction, not to expanded kinematic expressions. Fortran AmpliCol is the direct reference for zero, one and two quark-line colour matrices; beyond that, pyAmpliCol's generic open-line metric is validated through raw-amplitude probes and internal colour consistency checks.

4 Example: $d\bar{d} \to Zgg$

This section spells out one concrete DAG so that the split between compiled current contractions and Rust-interpreted scheduling is explicit. The user-level process is

$$d(p_1)\,\bar{d}(p_2) \to Z(p_3)\,g(p_4)\,g(p_5).$$

pyAmpliCol stores the process in the all-outgoing convention. Thus labels $1, \ldots, 5$ carry the fields

$$\bar{d}_1, \quad d_2, \quad Z_3, \quad g_4, \quad g_5,$$

with momenta $-p_1$, $-p_2$, p_3, p_4, and p_5, respectively. A label set such as $\{2, 4, 5\}$ is represented by $L = (2, 4, 5)$ and by the mask

$$m(2, 4, 5) = 2^{2-1} + 2^{4-1} + 2^{5-1} = 26.$$

The momentum mask is the same bit pattern in this example, and it tells Rusticol which stored momentum sum must be passed to the next compiled evaluator.

For one leading-color sector, the colored ordered labels are

$$O = (2, 4, 5, 1),$$

with the color-singlet Z_3 allowed to attach anywhere on the quark line. The color engine may also provide the sector $O = (2, 5, 4, 1)$. Nothing in the DAG builder special-cases this as a "$Z + 2g$" process: source currents are ordinary external-leg currents and all later currents are discovered from model vertices.

Source current indices

For a fixed retained source helicity choice, the source entries look as follows. The bit b_i denotes the unique helicity-source bit assigned to that external state.

source	current	mask	index payload
S_1	$\mathcal{J}_{\bar{d}}(\{1\})$	1	$(\text{pid} = -1, L = (1), H = b_1, \chi, S, F = (-1), Q, C_{\bar{q}}, P = 1)$
S_2	$\mathcal{J}_d(\{2\})$	2	$(\text{pid} = 1, L = (2), H = b_2, \chi, S, F = (1), Q, C_q, P = 2)$
S_3	$\epsilon_Z(\{3\})$	4	$(\text{pid} = 23, L = (3), H = b_3, S = h_Z, C_{\text{singlet}}, P = 4)$
S_4	$\epsilon_g(\{4\})$	8	$(\text{pid} = 21, L = (4), H = b_4, S = h_4, C_g, P = 8)$
S_5	$\epsilon_g(\{5\})$	16	$(\text{pid} = 21, L = (5), H = b_5, S = h_5, C_g, P = 16)$

The actual stored index also contains the exact leading-color sector id, line-group metadata, and the current dimension. For this example the quark and anti-quark source currents have Weyl-spinor components, gluons and the Z have Lorentz-vector components, and auxiliary tensor currents have the antisymmetric tensor components required by the cubicized four-gluon rule.

What each compiled stage computes

The generic stage compiler groups interactions by the size of the result current. For each group it emits a multi-output Symbolica evaluator. That evaluator is compiled once and receives only parent-current components, momentum sums, and real couplings as parameters. It returns the components of all result currents in the stage.

For the sector $O = (2, 4, 5, 1)$, the schematic stage contents are:

$$E_2: \quad \mathcal{J}_g(4,5) = P_g(p_{45})\, \mathcal{V}_{3g}\big(S_4, S_5; p_4, p_5\big), \tag{4}$$

$$\mathcal{J}_T(4,5) = \mathcal{V}_{ggT}\big(S_4, S_5\big), \tag{5}$$

$$\mathcal{J}_d(2,4) = P_q(p_{24})\, \mathcal{V}_{qgq}\big(S_2, S_4; p_2, p_4\big), \tag{6}$$

$$\mathcal{J}_d(2,3) = P_q(p_{23})\, \mathcal{V}_{qZq}\big(S_2, S_3; p_2, p_3\big), \tag{7}$$

plus the analogous allowed currents for the other color sector and for the anti-quark side. The notation E_2 means a compiled evaluator for all two-label result currents, not a Python function that loops over these formulas component by component.

The next stage consumes the rows written by E_2:

$$E_3: \quad \mathcal{J}_d(2,4,5) = P_q(p_{245}) \left[\mathcal{V}_{qgq}\big(\mathcal{J}_d(2,4), S_5\big) + \mathcal{V}_{qgq}\big(S_2, \mathcal{J}_g(4,5)\big)\right], \tag{8}$$

$$\mathcal{J}_d(2,3,4) = P_q(p_{234}) \left[\mathcal{V}_{qZq}\big(\mathcal{J}_d(2,4), S_3\big) + \mathcal{V}_{qgq}\big(\mathcal{J}_d(2,3), S_4\big)\right]. \tag{9}$$

The stage compiler does not expand these expressions into diagrams. It collects all interaction records whose output index is the same current and compiles the sum into the output components of that current. If two different splits produce the same current index, their contributions are accumulated inside the same compiled stage output.

The four-label stage then builds the endpoint current used by the closure:

$$E_4: \quad \mathcal{J}_d(2,3,4,5) = P_q(p_{2345}) \sum_{\text{compatible splits}} \mathcal{V}\big(\mathcal{J}_{\text{left}}, \mathcal{J}_{\text{right}}\big). \tag{10}$$

The sum includes, for example, gluon-first and Z-insertion contributions such as

$$\mathcal{V}_{qZq}\big(\mathcal{J}_d(2,4,5), S_3\big), \qquad \mathcal{V}_{qgq}\big(\mathcal{J}_d(2,3,4), S_5\big), \qquad \mathcal{V}_{qgq}\big(\mathcal{J}_d(2,3), \mathcal{J}_g(4,5)\big).$$

Those are still local vertex applications selected by the model and color engine. The result is one stored current row, identified by the full CurrentIndex and by its value-slot range in the artifact.

Finally, a compiled amplitude evaluator returns raw complex roots:

$$E_{\text{amp}}: \quad R_a = \mathcal{V}_{\text{close}}\big(\mathcal{J}_d(2,3,4,5), S_1\big), \tag{11}$$

with one output for each retained root. Roots with the same $(H, \text{color sector}, O)$ coherent-group key are summed before squaring. The reported leading-color matrix element is

$$|\mathcal{M}|^2_{\text{LC}} = \mathcal{N}_{\text{AC}} \sum_{\text{groups } g} w_g \left| \sum_{a \in g} R_a \right|^2. \tag{12}$$

The same example as a DAG

A stripped-down adjacency-list representation of this example is:

```
sources interpreted by Rusticol:
  S1 = J(pid=-1, labels={1}, H=b1, color=qbar-line)
  S2 = J(pid= 1, labels={2}, H=b2, color=q-line)
  S3 = J(pid=23, labels={3}, H=b3, color=singlet)
  S4 = J(pid=21, labels={4}, H=b4, color=line gluon)
  S5 = J(pid=21, labels={5}, H=b5, color=line gluon)

compiled stage E2, subset_size=2:
  I0: S4 + S5 -- V3g, Pg --> J(pid=21, labels={4,5})
  I1: S4 + S5 -- VggT    --> J(aux=T, labels={4,5})
  I2: S2 + S4 -- Vqgq,Pq --> J(pid=1, labels={2,4})
  I3: S2 + S3 -- VqZq,Pq --> J(pid=1, labels={2,3})

compiled stage E3, subset_size=3:
  I4: J(2,4) + S5     -- Vqgq,Pq --> J(pid=1, labels={2,4,5})
  I5: S2     + J(4,5) -- Vqgq,Pq --> J(pid=1, labels={2,4,5})
  I6: J(2,4) + S3     -- VqZq,Pq --> J(pid=1, labels={2,3,4})
  I7: J(2,3) + S4     -- Vqgq,Pq --> J(pid=1, labels={2,3,4})

compiled stage E4, subset_size=4:
  I8: J(2,4,5) + S3     -- VqZq,Pq --> J(pid=1, labels={2,3,4,5})
  I9: J(2,3,4) + S5     -- Vqgq,Pq --> J(pid=1, labels={2,3,4,5})
  I10:J(2,3)   + J(4,5) -- Vqgq,Pq --> J(pid=1, labels={2,3,4,5})

compiled amplitude stage:
  A0: J(pid=1, labels={2,3,4,5}) + S1 -- close --> R0
```

This listing is schematic: the real artifact contains all retained helicity ancestries, both leading-color sectors, source dimensions, output component ranges, momentum-slot ids, couplings, and color weights. The important point is the division of labor. The evaluator E_s performs the tensor contractions and the algebraic sum over interactions in stage s. Rusticol interprets the DAG schedule: fill sources, compute momentum sums, call E_2, scatter its outputs into current storage, call E_3, scatter again, and so on until E_{amp} and the final coherent square-sum.

5 Rusticol: the Implemented Rust Runtime

The previous staged pyAmpliCol runtime proved that the shared-current DAG could match Fortran AmpliCol numerically, but its Python loop still moved current tables between many evaluator calls. The current implementation

keeps Python as the process generator and validation driver, but moves the hot eager-DAG sweep into a PyO3 extension named `rusticol`. The generated artifact is now a self-contained process directory used by both runtimes:

```
rt_py = pyamplicol.load_process(process_dir, runtime="python")
values_py = rt_py.evaluate(momenta)

import rusticol
rt = rusticol.Runtime.load(process_dir)
values = rt.evaluate(momenta)
profile = rt.profile(momenta)
```

The Python and Rust runtimes read the same `process_manifest.json`, the same Symbolica evaluator manifests, and the same validation momenta. The Python path is kept as a transparent reference implementation. The Rust path is the performance path.

A process directory contains four classes of data. First, it records process and model metadata: external PDG order, momentum conventions, particle and vertex tables, couplings, masses, symmetry factors, and leading-color normalization. Second, it stores the shared-current DAG: source currents, current ids, dimensions, interaction stages, output slots, retained amplitude records, helicity weights, and color factors. Third, it stores the compiled Symbolica evaluator artifacts for each stage and for the final amplitude/raw-sum stage. Fourth, it contains standalone validation assets: `validation_momenta.json` with decimal-string momenta and a `check_standalone.py` script that imports only `rusticol`, plus `numpy` when available for the optimized f64 input path.

```
# Generated by pyAmpliCol.
process_dir/
  process_manifest.json
  manifest.json
  validation_momenta.json
  check_standalone.py
  compiled/              # C++/ASM shared libraries when present
  ... evaluator states ...
```

`rusticol.Runtime.load` streams the typed schema directly from disk, validates current/value relationships through precomputed indices, loads every Symbolica evaluator, computes direct state offsets for stage outputs, allocates reusable scratch buffers, and builds a fixed execution plan. The optimized f64 path accepts NumPy arrays with shape `(batch, n_external, 4)`. For each batch, Rust fills source wavefunctions and momentum sums, executes stage evaluators in topological order, writes the returned current components into the contiguous state table, executes the amplitude/raw-sum evaluator, applies the Fortran-matched normalization, and returns a NumPy array of matrix elements.

On AArch64, the f64 route loads native two-lane complex SymJIT payloads. A stage-local input slice is packed once and reused by every output chunk in that stage; this keeps chunking as a code-size control without multiplying input-pack cost. Setting `RUSTICOL_NATIVE_SIMD_JIT=0` selects the scalar JIT fallback for diagnostics. Runnable artifacts likewise omit the full per-interaction diagnostic records after evaluator serialization, retaining compact IDs and per-kind counts. The trusted local diagnostic sidecar is a gzip-compressed Python protocol-5 pickle; it is not part of evaluator loading or execution.

```
for point in batch {
    fill_source_currents(state_row, point);
    fill_momentum_sums(state_row, point);
}

for stage in stages {
    evaluated = stage.evaluator.evaluate_batch(state);
    assign_stage_outputs(state, evaluated, stage.output_offsets);
}

raw = amplitude_stage.evaluate_raw_sums(state);
me2 = normalization * raw;
```

The `profile` API reports the same split used in the benchmarks: source fill, momentum setup, stage evaluator time, output assignment, amplitude evaluator time, final reduction, total wall time, and memory snapshots where the platform exposes them.

The precision API follows Symbolica's model. `evaluate` and `evaluate_with_prec(..., 16)` use the specialized native f64 path. `evaluate_with_prec(..., 32)` maps the serialized exact evaluator state to Symbolica's `DoubleFloat` coefficients. Other positive precisions map to arbitrary-precision `Float` evaluators. The high-precision routes share the same stage orchestration code through generic Rust traits, while the f64 route remains specialized and avoids high-precision allocation overhead.

The artifact schema is process-generic. The staged execution format no longer needs a Rust-side process-family branch: each process directory supplies the source layout, current slots, stage records, amplitude roots, coherent groups, and evaluator artifacts needed by Rusticol. If a model vertex or color rule is not implemented yet, generation fails with an explicit unsupported-kernel diagnostic rather than falling back to a hidden family path.

6 Shared LC Ordering Recycling

LC process generation now has two related modes. For ordinary integration benchmarks, pyAmpliCol times one selected LC ordering per phase-space point. The selected artifact is pruned before Symbolica evaluator construction, so no dynamic colour-flow selector remains in those expressions. For all-flow fixed-helicity matrix entries, pyAmpliCol writes a materialized shared all-sector artifact: internal currents live in one shared LC sector, while amplitude roots retain the full colour-order mapping used for the AmpliCol `imode=2` comparison. This avoids the old sector-local all-ordering clone while keeping the runtime result directly value-comparable.

The shared all-sector mode reuses pure-gluon subcurrents through the usual LC reflection relation, with the corresponding sign carried as an interaction colour weight. That sign is applied before current-stage propagation, so the shared DAG is numerically equivalent to the sector-specialized artifact while avoiding one current table per ordering. Runtime selection of one LC ordering is still available through generated sidecars; `time-process -lc-sector-ids` loads the matching pruned sidecar when it exists.

The fixed-helicity workload is selected without constructing an all-helicity all-sector artifact. The matrix driver derives replay-compatible candidates from the external source-spin states, evaluates their selected-sector raw sums at a deterministic point, and keeps the strongest nonzero candidate. Exact sector-id lookup is indexed, so this setup remains linear in the number of LC sectors rather than repeatedly scanning the complete colour plan.

Process	Orders	Curr.	AmpliCol/pyAmpliCol O1 gen	O1 JIT frac.	Sel. ratio	AmpliCol/pyAmpliCol all run	Rel. diff.
$gg \to t\bar{t}+g$	6	29	0.0967 / 0.172	0.328	1.85×	1.45 / 0.738	1.4×10^{-16}
$gg \to t\bar{t}+2g$	24	118	0.105 / 0.267	0.587	1.47×	4.35 / 2.74	2.15×10^{-15}
$gg \to t\bar{t}+3g$	120	592	0.13 / 1.22	0.772	1.25×	25.4 / 31.6	0
$gg \to t\bar{t}+4g$	720	3554	0.412 / 10	0.826	1.41×	199 / 290	4.77×10^{-16}
$gg \to t\bar{t}+5g$	5040	24880	5.91 / 123	0.874	1.37×	2.85×10^3 / 3.45×10^3	6.23×10^{-15}
$gg \to t\bar{t}+6g$	40320	199042	272 / 1.26×10^3	0.862	1.68×	5.29×10^4 / 6.21×10^4	7.57×10^{-16}

The generation column reports AmpliCol/pyAmpliCol all-flow setup time in seconds for the fixed-helicity `imode=2`-style comparison. All run entries are wall microseconds per phase-space point, AmpliCol/pyAmpliCol. The selected-ratio column is the pyAmpliCol/AmpliCol multiplier for one LC flow with the usual helicity sum; the all-run column is the fixed-source-helicity sum over all LC flows. The generated table is refreshed from the LC matrix cache. At $gg \to t\bar{t}+6g$, both implementations report 199,042 filtered currents and 40,320 amplitudes/orderings; the selected-flow validation agrees to better than 10^{-15}. The process-matrix tables below use the same selected- and all-flow runtime semantics.

7 Generic LC Performance Matrix (SymJIT O1; O3*)

Cell format: generation time above runtime per phase-space point; slots are AmpliCol, pyAmpliCol SymJIT O1 selected flow (O3 when starred), and pyAmpliCol SymJIT O1 all flows. AmpliCol uses generated-library timings. Conventions and gaps are summarized after the table.

ID	base process	n=1				n=2				n=3			
1	$d\bar{d} \to Z + (n-1)g$	1.873 0.0193	/ 0.101 / 0.859	(x0.099) (x8.32)	(x1.34) (x0.159)	1.996 0.115	/ 0.102 / 0.968	(x0.113) (x2.94)	(x1.41) (x0.205)	1.958 0.424	/ 0.0944 / 1.144	(x0.154) (x1.82)	(x1.64) (x0.325)
2	$u\bar{d} \to W^+ + (n-1)g$	1.886 0.0119	/ 0.0989 / 0.863	(x0.092) (x11.1)	(x1.41) (x0.158)	1.986 0.0717	/ 0.102 / 0.9	(x0.103) (x3.36)	(x1.51) (x0.224)	1.938 0.232	/ 0.0973 / 1.177	(x0.13) (x2.25)	(x1.64) (x0.313)
3	$d\bar{d} \to e^+e^- + (n-2)g$	N/A				2.016 0.0679	/ 0.101 / 0.938	(x0.102) (x3.01)	(x1.50) (x0.207)	1.985 0.154	/ 0.0953 / 1.076	(x0.116) (x2.39)	(x1.63) (x0.268)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	N/A				2.028 0.0335	/ 0.103 / 1.019	(x0.094) (x3.75)	(x1.48) (x0.186)	1.914 0.0613	/ 0.0981 / 0.957	(x0.108) (x3.46)	(x1.72) (x0.284)
5	$d\bar{d} \to ZZ + (n-2)g$	N/A				2.064 0.165	/ 0.102 / 0.983	(x0.136) (x2.54)	(x1.50) (x0.208)	2.026 0.693	/ 0.0976 / 1.049	(x0.23) (x1.58)	(x1.61) (x0.314)
6	$gg \to t\bar{t} + (n-2)g$	N/A				2.058 0.102	/ 0.103 / 1.069	(x0.135) (x4.20)	(x1.52) (x0.254)	2.283 0.646	/ 0.0967 / 1.454	(x0.176) (x1.85)	(x1.77) (x0.507)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	N/A				2.025 0.0513	/ 0.103 / 1.01	(x0.098) (x4.70)	(x1.48) (x0.241)	2.031 0.234	/ 0.0965 / 1.24	(x0.125) (x2.29)	(x1.76) (x0.434)
8	$gg \to gg + (n-2)g$	N/A				1.916 0.158	/ 0.0937 / 1.03	(x0.118) (x2.18)	(x1.69) (x0.309)	2.117 0.855	/ 0.104 / 2.394	(x0.265) (x1.40)	(x1.94) (x0.499)
9	$d\bar{d} \to ZZZ + (n-3)g$	N/A				N/A				2.424 1.477	/ 0.0988 / 1.249	(x0.379) (x1.49)	(x1.58) (x0.322)
10	$d\bar{d} \to e^+e^- ZH + (n-4)g$	N/A				N/A				N/A			
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	N/A				N/A				N/A			
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	N/A				N/A				N/A			
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	N/A				N/A				N/A			
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A				N/A				N/A			
gen JIT one-flow, hel. sum		1.87| x0.092	1.89| x0.099	1.88| x0.096	1.88 x0.096	1.92| x0.094	2.06| x0.136	2.01| x0.112	2.02 x0.108	x0.112	1.91| x0.108	2.42| x0.379	2.08| x0.187	2.03 x0.154	x0.192
gen O1 all-flows, one-hel		0.0989| x1.34	0.101| x1.41	0.1| x1.37	0.1 x1.37	0.0937| x1.41	0.103| x1.69	0.101| x1.51	0.102 x1.50	x1.51	0.0944| x1.58	0.104| x1.94	0.0976| x1.70	0.0973 x1.64	x1.70
run JIT one-flow, hel. sum		0.0119| x8.32	0.0193| x11.1	0.0156| x9.74	0.0156 x9.74	0.0335| x2.18	0.165| x4.70	0.0955| x3.34	0.0866 x3.18	x3.06	0.0613| x1.40	1.48| x3.46	0.531| x2.06	0.424 x1.85	x1.70
run O1 all-flows, one-hel		0.859| x0.158	0.863| x0.159	0.861| x0.158	0.861 x0.158	0.9| x0.186	1.07| x0.309	0.99| x0.229	0.996 x0.216	x0.23	0.957| x0.268	2.39| x0.507	1.3| x0.363	1.18 x0.322	x0.383

Generic LC Performance Matrix (SymJIT O1; O3*) (continued)

ID	base process	n=4				n=5				n=6			
1	$d\bar{d} \to Z + (n-1)g$	2.056 1.371	/ 0.0961 / 1.954	(x0.369) (x1.41)	(x1.79) (x0.472)	2.32 4.356	/ 0.109 / 7.354	(x0.795) (x1.19)	(x2.54) (x0.431)	3.186 13.93	/ 0.213 / 41.84	(x1.28) (x1.15)	(x5.82) (x0.757)
2	$u\bar{d} \to W^+ + (n-1)g$	2.003 0.864	/ 0.097 / 2.048	(x0.247) (x1.54)	(x1.79) (x0.445)	2.138 2.801	/ 0.108 / 7.252	(x0.57) (x1.32)	(x2.58) (x0.425)	2.495 9.301	/ 0.22 / 43.76	(x1.18) (x1.13)	(x5.62) (x0.733)
3	$d\bar{d} \to e^+e^- + (n-2)g$	2.055 0.44	/ 0.097 / 1.228	(x0.174) (x1.69)	(x1.68) (x0.383)	2.196 1.32	/ 0.106 / 2.24	(x0.264) (x1.35)	(x1.68) (x0.473)	2.493 3.995	/ 0.172 / 8.14	(x0.57) (x1.16)	(x1.68) (x0.424)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	2 0.155	/ 0.101 / 1.178	(x0.146) (x2.72)	(x1.54) (x0.374)	2.119 0.645	/ 0.126 / 2.173	(x0.19) (x1.63)	(x1.41) (x0.472)	2.208 2.123	/ 0.157 / 7.369	(x0.425) (x1.37)	(x1.80) (x0.452)
5	$d\bar{d} \to ZZ + (n-2)g$	2.319 2.073	/ 0.103 / 1.557	(x0.441) (x1.27)	(x1.58) (x0.385)	2.578 6.281	/ 0.115 / 3.411	(x0.905) (x1.05)	(x1.71) (x0.449)	3.521 17.1	/ 0.19 / 13.52	(x1.29) (x1.15)	(x2.04) (x0.381)
6	$gg \to t\bar{t} + (n-2)g$	3.189 2.352	/ 0.105 / 4.349	(x0.369) (x1.47)	(x2.54) (x0.631)	6.277 7.8	/ 0.13 / 25.36	(x0.471) (x1.25)	(x9.39) (x1.25)	25.49 22.51	/ 0.412 / 199	(x0.241) (x1.41)	(x24.3) (x1.46)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	2.274 0.864	/ 0.104 / 2.456	(x0.229) (x1.63)	(x2.10) (x0.607)	2.971 2.839	/ 0.124 / 10.48	(x0.439) (x1.39)	(x4.39) (x0.575)	4.123 8.926	/ 0.233 / 65.35	(x0.78) (x1.23)	(x13.9) (x1.10)
8	$gg \to gg + (n-2)g$	2.182 3.178	/ 0.11 / 13.22	(x0.597) (x1.26)	(x5.55) (x1.01)	2.829 10.32	/ 0.198 / 105	(x1.08) (x1.18)	(x23.9) (x1.26)	5.012 32.19	/ 3.471 / 977	(x1.36) (x1.22)	(x15.2) (x1.60)
9	$d\bar{d} \to ZZZ + (n-3)g$	2.97 4.665	/ 0.102 / 1.548	(x0.685) (x1.18)	(x1.53) (x0.401)	4.709 13	/ 0.124 / 2.504	(x0.798) (x1.05)	(x1.41) (x0.464)	8.469 33.08	/ 0.208 / 6.732	(x0.864) (x1.16)	(x1.21) (x0.438)
10	$d\bar{d} \to e^+e^- ZH + (n-4)g$	3.694 0.742	/ 0.0989 / 1.459	(x0.192) (x2.08)	(x1.66) (x0.427)	4.471 1.581	/ 0.115 / 1.662	(x0.211) (x1.74)	(x1.48) (x0.532)	5.312 3.787	/ 0.166 / 2.579	(x0.289) (x1.32)	(x1.12) (x0.573)
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	2.945 1.043	/ 0.103 / 1.547	(x0.218) (x1.66)	(x1.68) (x0.606)	4.94 3.329	/ 0.135 / 2.811	(x0.31) (x1.42)	(x1.77) (x0.769)	10.1 11.48	/ 0.238 / 8.833	(x0.398) (x1.35)	(x2.30) (x0.725)
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	3.879 1.394	/ 0.106 / 1.691	(x0.145) (x1.06)	(x1.55) (x0.3)	4.611 2.763	/ 0.117 / 2.026	(x0.2) (x0.949)	(x1.39) (x0.346)	5.585 6.406	/ 0.167 / 2.974	(x0.271) (x0.734)	(x1.06) (x0.387)
13	$d\bar{d} \to u\bar{u}\, s\bar{s} + (n-4)g$	2.188 0.173		(x0.107) (x2.14)	0.234 (N/A)	3.18 0.662		(x0.102) (x1.26)	0.324 (N/A)	5.499 2.289		(x0.129) (x0.882)	0.708 (N/A)
14	$d\bar{d} \to u\bar{u}\, s\bar{s}\, c\bar{c} + (n-6)g$	N/A				N/A				N/A N/A		2.149 (5.96)	2.149 N/A

gen JIT one-flow, hel. sum	2|	3.88|	2.6|	2.27		2.12|	6.28|	3.49|	2.97		2.21|	25.5|	6.42|	5.01	
	x0.107|	x0.685|	x0.301|	x0.229|	x0.299	x0.102|	x1.08|	x0.487|	x0.439|	x0.467	x0.129|	x1.36|	x0.698|	x0.57|	x0.541
gen O1 all-flows, one-hel	0.0961|	0.11|	0.102|	0.102		0.106|	0.198|	0.126|	0.121		0.157|	3.47|	0.487|	0.21	
	x1.53|	x5.55|	x2.08|	x1.68|	x2.11	x1.39|	x23.9|	x4.47|	x1.74|	x5.54	x1.06|	x24.3|	x6.34|	x2.17|	x12.1
run JIT one-flow, hel. sum	0.155|	4.67|	1.49|	1.04		0.645|	13|	4.44|	2.84		2.12|	33.1|	12.9|	9.3	
	x1.06|	x2.72|	x1.62|	x1.54|	x1.38	x0.949|	x1.74|	x1.29|	x1.26|	x1.19	x0.734|	x1.41|	x1.17|	x1.16|	x1.20
run O1 all-flows, one-hel	1.18|	13.2|	2.85|	1.62		1.66|	105|	14.4|	3.11		2.58|	977|	115|	11.2	
	x0.3|	x1.01|	x0.503|	x0.436|	x0.688	x0.346|	x1.26|	x0.62|	x0.473|	x1.07	x0.381|	x1.60|	x0.753|	x0.649|	x1.46

Generic LC Performance Matrix (SymJIT O1; O3*) (continued)

ID	base process	n=7				n=8				n=9			
1	$d\bar{d} \to Z + (n-1)g$	4.109 34.92	/ 1.167 / 287	(x1.96) (x1.26)	(x8.17) (x1.06)	8.81 90.29	/ 16.39 / 5e+03	(x1.89) (x1.16)	(x7.45) (x0.643)	24.92 226	/ 517 / 8.89e+04	(x1.59) (x1.33)	(x2.70) (x0.724)
2	$u\bar{d} \to W^+ + (n-1)g$	3.031 25.54	/ 1.164 / 297	(x2.15) (x1.29)	(x8.42) (x1.03)	4.618 68	/ 16.03 / 5.34e+03	(x2.89) (x1.21)	(x7.63) (x0.608)	N/A			
3	$d\bar{d} \to e^+e^- + (n-2)g$	2.67 11.65	/ 0.663 / 45.42	(x1.33) (x1.10)	(x1.95) (x0.73)	3.701 32.19	/ 6.831 / 322	(x2.07) (x1.25)	(x1.47) (x1.04)	N/A			
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	2.282 6.815	/ 0.653 / 42.52	(x0.968) (x1.20)	(x1.91) (x0.688)	2.655 20.38	/ 6.764 / 302	(x1.98) (x1.32)	(x1.43) (x1.01)	N/A			
5	$d\bar{d} \to ZZ + (n-2)g$	6.452 43.37	/ 0.773 / 77.24	(x1.32) (x1.15)	(x2.51) (x0.719)	16.52 106	/ 7.949 / 539	(x1.08) (x1.07)	(x1.97) (x0.917)	N/A			
6	$gg \to t\bar{t} + (n-2)g$	221 59.82	/ 5.905 / 2.85e+03	(x0.059) (x1.37)	(x20.8) (x1.21)	2.3e+03 153	/ 272 / 5.29e+04	(x0.013) (x1.68)	(x4.63) (x1.17)	N/A			
7	$d\bar{d} \to t\bar{t} + (n-2)g$	10.68 25.31	/ 1.369 / 471	(x0.638) (x1.41)	(x23.4) (x1.35)	70.48 70.57	/ 25.68 / 9.94e+03	(x0.219) (x1.26)	(x13.5) (x0.874)	N/A			
8	$gg \to gg + (n-2)g$	10.5 82.77	/ 232 / 2.62e+04	(x1.52) (x1.38)	(x7.55) (x1.20)	51.71 209	/ 2.48e+04 / 3.81e+05	(x0.799) (x1.44)	>30 GB RAM >30 GB RAM	N/A			
9	$d\bar{d} \to ZZZ + (n-3)g$	22.21 80.67	/ 0.867 / 28.11	(x0.644) (x1.11)	(x0.744) (x0.359)	66.46 186	/ 6.915 / 153	(x0.481) (x1.00)	(x0.524) (x0.768)	N/A			
10	$d\bar{d} \to e^+e^- ZH + (n-4)g$	7.707 9.804	/ 0.711 / 6.02	(x0.4) (x1.21)	(x0.351) (x0.495)	14.08 24.15	/ 6.051 / 24.49	(x0.467) (x1.25)	(x0.097) (x0.369)	N/A			
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	31.04 31.54	/ 0.989 / 44.41	(x0.291) (x1.44)	(x2.39) (x1.17)	134 85.72	/ 7.424 / 263	(x0.156) (x1.37)	(x2.17) (x1.31)	N/A			
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	8.688 16.34	/ 0.719 / 7.102	(x0.318) (x0.624)	(x0.31) (x0.319)	16.2 36.43	/ 6.069 / 27.39	(x0.352) (x0.738)	(x0.076) (x0.244)	N/A			
13	$d\bar{d} \to u\bar{u}\, s\bar{s} + (n-4)g$	13.03 7.384		(x0.128) (x0.706)	1.661 (N/A)	35.07 21.44		(x0.117) (x0.779)	4.094 (N/A)	N/A			
14	$d\bar{d} \to u\bar{u}\, s\bar{s}\, c\bar{c} + (n-6)g$	N/A N/A		11.25 (72.8)	11.25 N/A	N/A N/A		>30 GB RAM >30 GB RAM	>30 GB RAM >30 GB RAM	N/A			

gen JIT one-flow, hel. sum	2.28|	221|	26.4|	8.69		2.66|	2.3e3|	209|	16.5		24.9|	24.9|	24.9|	24.9	
	x0.059|	x2.15|	x0.902|	x0.644|	x0.278	x0.013|	x2.89|	x0.964|	x0.481|	x0.08	x1.59|	x1.59|	x1.59|	x1.59|	x1.59
gen O1 all-flows, one-hel	0.653|	232|	20.5|	0.928		6.05|	2.48e4|	2.1e3|	7.69		517|	517|	517|	517	
	x0.31|	x23.4|	x6.54|	x2.45|	x7.84	x0.002|	x13.5|	x3.41|	x1.72|	x0.077	x2.70|	x2.70|	x2.70|	x2.70|	x2.70
run JIT one-flow, hel. sum	6.82|	82.8|	33.5|	25.5		20.4|	209|	84.8|	70.6		226|	226|	226|	226	
	x0.624|	x1.44|	x1.17|	x1.21|	x1.24	x0.738|	x1.68|	x1.19|	x1.25|	x1.26	x1.33|	x1.33|	x1.33|	x1.33|	x1.33
run O1 all-flows, one-hel	6.02|	2.62e4|	2.53e3|	61.3		24.5|	3.81e5|	3.8e4|	431		8.89e4|	8.89e4|	8.89e4|	8.89e4	
	x0.319|	x1.35|	x0.861|	x0.881|	x1.20	x0.244|	x1.31|	x0.814|	x0.874|	x1.05	x0.724|	x0.724|	x0.724|	x0.724|	x0.724

Each AmpliCol pair is selected-flow/helicity-summed then all-flow/fixed-helicity; pyAmpliCol SymJIT O1 uses matching pruned and shared artifacts. Entries are generation seconds above runtime wall microseconds per point; pyAmpliCol-only cells are absolute. Green/orange/red means below one/below two/at least two. Summaries are `min|max|avg|med|sum`; `sum` compares paired totals. A superscript * marks a selected-flow JIT O3 override; `>30 GB RAM` marks only the affected selected/all-flow workload. A **VALIDATION FAILED** marker means the same-point difference exceeds 10^{-8}.

Settings. `docs/result_matrix.py` (`-color-accuracy lc`) writes this table and its JSON cache. AmpliCol uses its generated library for the selected workload and the fixed-helicity `imode=2` colour probe for all flows. pyAmpliCol writes separate `selected_flow` and shared `all_flows` artifacts. pyAmpliCol uses Rusticol double precision; selected/all-flow batches are 128/64. Selected JIT stages measure chunks around base 128; all-flow outputs use uniform chunk 8192. Stage-local inputs, ten Horner iterations, default CPE rounds, expanded optimization limits, SymJIT O1 by default with starred selected-flow O3 overrides, and `-target-runtime 10` are used. The two LC workloads are displayed side by side.

Runtime interpretation. On AArch64, Rusticol executes saved SymJIT stages as native two-lane complex evaluators and packs each stage input once for all of its output chunks. At very low multiplicity, source filling, stage transfer, and result reduction can still dominate a wall-time comparison against an AmpliCol call measured in tens of nanoseconds, even when the pure evaluator is faster. The evaluator-dominated $n \geq 5$ cells are therefore the runtime-optimization gate.

8 Dedicated $d\bar{d} \to Z+$ Gluon Performance

n	process	route	setup	selected flow, helicity sum gen [s]	selected flow, helicity sum wall [us/pt]	selected flow, helicity sum eval [us/pt]	all flows, fixed helicity gen [s]	all flows, fixed helicity wall [us/pt]	all flows, fixed helicity eval [us/pt]	notes
1	d d~ > Z + (1-1)*g	reference	AmpliCol	1.87	N/A	0.0193	0.101	N/A	0.859	
1	d d~ > Z + (1-1)*g	Rusticol	pyAmpliCol JIT O1	0.145 (x0.08)	0.185 (x9.58)	0.0535 (x2.77)	0.135 (x1.34)	0.136 (x0.16)	0.0327 (x0.04)	
1	d d~ > Z + (1-1)*g	Rusticol	pyAmpliCol ASM O3	0.799 (x0.43)	0.221 (x11.5)	0.0918 (x4.76)	0.635 (x6.28)	0.139 (x0.16)	0.0314 (x0.04)	
1	d d~ > Z + (1-1)*g	Rusticol	pyAmpliCol C++ O3	0.694 (x0.37)	0.172 (x8.94)	0.0415 (x2.15)	0.658 (x6.50)	0.133 (x0.15)	0.0254 (x0.03)	
1	d d~ > Z + (1-1)*g	Rusticol	pyAmpliCol JIT O3	0.148 (x0.08)	0.178 (x9.22)	0.0473 (x2.45)	0.138 (x1.36)	0.14 (x0.16)	0.0336 (x0.04)	
2	d d~ > Z + (2-1)*g	reference	AmpliCol	2	N/A	0.115	0.102	N/A	0.968	
2	d d~ > Z + (2-1)*g	Rusticol	pyAmpliCol JIT O1	0.143 (x0.07)	0.373 (x3.24)	0.195 (x1.70)	0.144 (x1.41)	0.198 (x0.20)	0.0771 (x0.08)	
2	d d~ > Z + (2-1)*g	Rusticol	pyAmpliCol ASM O3	0.955 (x0.48)	0.73 (x6.34)	0.549 (x4.76)	0.914 (x8.99)	0.269 (x0.28)	0.143 (x0.15)	
2	d d~ > Z + (2-1)*g	Rusticol	pyAmpliCol C++ O3	1.21 (x0.61)	0.41 (x3.56)	0.229 (x1.99)	0.951 (x9.35)	0.194 (x0.20)	0.0688 (x0.07)	
2	d d~ > Z + (2-1)*g	Rusticol	pyAmpliCol JIT O3	0.178 (x0.09)	0.38 (x3.30)	0.2 (x1.74)	0.149 (x1.47)	0.204 (x0.21)	0.0799 (x0.08)	
3	d d~ > Z + (3-1)*g	reference	AmpliCol	1.96	N/A	0.424	0.0944	N/A	1.14	
3	d d~ > Z + (3-1)*g	Rusticol	pyAmpliCol JIT O1	0.17 (x0.09)	0.806 (x1.90)	0.54 (x1.28)	0.155 (x1.64)	0.371 (x0.32)	0.208 (x0.18)	
3	d d~ > Z + (3-1)*g	Rusticol	pyAmpliCol ASM O3	1.3 (x0.66)	1.93 (x4.56)	1.67 (x3.93)	1.18 (x12.5)	0.681 (x0.60)	0.518 (x0.45)	
3	d d~ > Z + (3-1)*g	Rusticol	pyAmpliCol C++ O3	2.31 (x1.18)	0.961 (x2.27)	0.694 (x1.64)	1.34 (x14.2)	0.369 (x0.32)	0.208 (x0.18)	
3	d d~ > Z + (3-1)*g	Rusticol	pyAmpliCol JIT O3	0.226 (x0.12)	0.806 (x1.90)	0.539 (x1.27)	0.153 (x1.62)	0.366 (x0.32)	0.205 (x0.18)	
4	d d~ > Z + (4-1)*g	reference	AmpliCol	2.06	N/A	1.37	0.0961	N/A	1.95	
4	d d~ > Z + (4-1)*g	Rusticol	pyAmpliCol JIT O1	0.221 (x0.11)	2.01 (x1.46)	1.56 (x1.14)	0.172 (x1.79)	0.923 (x0.47)	0.667 (x0.34)	
4	d d~ > Z + (4-1)*g	Rusticol	pyAmpliCol ASM O3	1.79 (x0.87)	5.48 (x4.00)	5.03 (x3.67)	1.49 (x15.5)	2.23 (x1.14)	1.97 (x1.01)	
4	d d~ > Z + (4-1)*g	Rusticol	pyAmpliCol C++ O3	5.45 (x2.65)	2.7 (x1.97)	2.25 (x1.64)	2.29 (x23.9)	1.03 (x0.53)	0.773 (x0.40)	
4	d d~ > Z + (4-1)*g	Rusticol	pyAmpliCol JIT O3	0.357 (x0.17)	2 (x1.46)	1.55 (x1.13)	0.173 (x1.81)	0.919 (x0.47)	0.665 (x0.34)	
5	d d~ > Z + (5-1)*g	reference	AmpliCol	2.32	N/A	4.36	0.109	N/A	7.35	
5	d d~ > Z + (5-1)*g	Rusticol	pyAmpliCol JIT O1	0.364 (x0.16)	6.07 (x1.39)	5.09 (x1.17)	0.277 (x2.54)	3.17 (x0.43)	2.62 (x0.36)	
5	d d~ > Z + (5-1)*g	Rusticol	pyAmpliCol ASM O3	3.86 (x1.67)	16.1 (x3.69)	15.1 (x3.46)	2.06 (x18.9)	9.36 (x1.27)	8.81 (x1.20)	
5	d d~ > Z + (5-1)*g	Rusticol	pyAmpliCol C++ O3	16.3 (x7.01)	8.01 (x1.84)	7.02 (x1.61)	8.06 (x74)	4.32 (x0.59)	3.77 (x0.51)	
5	d d~ > Z + (5-1)*g	Rusticol	pyAmpliCol JIT O3	0.671 (x0.29)	6.02 (x1.38)	5.04 (x1.16)	0.278 (x2.55)	3.17 (x0.43)	2.62 (x0.36)	
6	d d~ > Z + (6-1)*g	reference	AmpliCol	3.19	N/A	13.9	0.213	N/A	41.8	
6	d d~ > Z + (6-1)*g	Rusticol	pyAmpliCol JIT O1	0.735 (x0.23)	16.2 (x1.16)	14.2 (x1.02)	1.24 (x5.82)	31.7 (x0.76)	29.5 (x0.70)	
6	d d~ > Z + (6-1)*g	Rusticol	pyAmpliCol ASM O3	7.43 (x2.33)	45 (x3.23)	42.9 (x3.08)	4.08 (x19.1)	66 (x1.58)	63.6 (x1.52)	
6	d d~ > Z + (6-1)*g	Rusticol	pyAmpliCol C++ O3	45.9 (x14.4)	25 (x1.79)	22.9 (x1.65)	67.5 (x316)	52 (x1.24)	49.8 (x1.19)	
6	d d~ > Z + (6-1)*g	Rusticol	pyAmpliCol JIT O3	1.47 (x0.46)	16 (x1.15)	14.2 (x1.02)	1.25 (x5.84)	32.8 (x0.78)	30.6 (x0.73)	
7	d d~ > Z + (7-1)*g	reference	AmpliCol	4.11	N/A	34.9	1.17	N/A	287	
7	d d~ > Z + (7-1)*g	Rusticol	pyAmpliCol JIT O1	1.7 (x0.41)	50.8 (x1.46)	47.1 (x1.35)	9.54 (x8.17)	303 (x1.06)	296 (x1.03)	
7	d d~ > Z + (7-1)*g	Rusticol	pyAmpliCol ASM O3	15.3 (x3.72)	113 (x3.24)	109 (x3.12)	20.2 (x17.3)	543 (x1.89)	535 (x1.87)	
7	d d~ > Z + (7-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A	
7	d d~ > Z + (7-1)*g	Rusticol	pyAmpliCol JIT O3	4.18 (x1.02)	50.8 (x1.46)	47.3 (x1.35)	9.8 (x8.40)	303 (x1.06)	296 (x1.03)	
8	d d~ > Z + (8-1)*g	reference	AmpliCol	8.81	N/A	90.3	16.4	N/A	5e3	
8	d d~ > Z + (8-1)*g	Rusticol	pyAmpliCol JIT O1	4.36 (x0.50)	168 (x1.86)	162 (x1.79)	122 (x7.45)	3.22e3 (x0.64)	3.16e3 (x0.63)	
8	d d~ > Z + (8-1)*g	Rusticol	pyAmpliCol ASM O3	34.3 (x3.89)	298 (x3.30)	292 (x3.24)	211 (x12.8)	5.31e3 (x1.06)	5.24e3 (x1.05)	
8	d d~ > Z + (8-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A	
8	d d~ > Z + (8-1)*g	Rusticol	pyAmpliCol JIT O3	9.4 (x1.07)	168 (x1.86)	162 (x1.79)	123 (x7.49)	3.22e3 (x0.64)	3.16e3 (x0.63)	
9	d d~ > Z + (9-1)*g	reference	AmpliCol	24.9	N/A	226	517	N/A	8.89e4	
9	d d~ > Z + (9-1)*g	Rusticol	pyAmpliCol JIT O1	13.7 (x0.55)	329 (x1.45)	305 (x1.35)	1.4e3 (x2.70)	6.44e4 (x0.72)	6.34e4 (x0.71)	reused matching O1 all-flow artifact from LC result matrix cache; time batch 64, output chunk 8192
9	d d~ > Z + (9-1)*g	Rusticol	pyAmpliCol ASM O3	70 (x2.81)	901 (x3.98)	887 (x3.92)	1.98e3 (x3.83)	1.1e5 (x1.24)	1.1e5 (x1.23)	
9	d d~ > Z + (9-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A	
9	d d~ > Z + (9-1)*g	Rusticol	pyAmpliCol JIT O3	13.9 (x0.56)	379 (x1.67)	353 (x1.56)	1.4e3 (x2.72)	6.02e4 (x0.68)	5.92e4 (x0.67)	

Final-state multiplicity n means $d\bar{d} \to Z + (n-1)g$. pyAmpliCol rows use Rusticol; selected-flow timings use batch size 64 and uniform output chunk size 128, while all-flow fixed-helicity timings use batch size 64 and output chunk size 8192. Both use stage-local evaluator inputs, ten Horner iterations, and `time-process -target-runtime 10`. The first timing block is one selected LC flow with the helicity sum; the second is the sum over all LC flows for one fixed source helicity. Generation columns report the artifact used by the corresponding timing block: selected-flow generation on the left and all-flow generation on the right. The green row is the default pyAmpliCol route, SymJIT O3. Generated processes are kept under `docs/.z_performance_outputs`; the driver is `docs/run_z_performance_table.py`. On AArch64, the ASM row uses Symbolica's scalar inline-assembly export; the production SymJIT rows use the native two-lane complex path, so ASM is retained as a diagnostic backend rather than the expected runtime optimum.

To reproduce one pyAmpliCol row, run generation followed by timing. This example is the $n = 7$ JIT O3 selected-flow block; for the all-flow block, use the same command but use `-symbolica-output-chunk-size 8192` and replace the sector/order flags with `-lc-sector-strategy all -skip-generic-plan -no-runtime-lc-sector-selector -source-helicities 1=-1,2=1,3=-1,4=1,5=1,6=1,7=1,8=1,9=1`; then time it with `-batch-size 64`.

```
env PYTHONPATH=src dependencies/.venv/bin/python -m pyamplicol generate-process 'd d~ > z g g g g g g' docs/.z_performance_outputs/manual/n7/jit_o3 --replace --n_cores 5 --color-accuracy lc --batch-size 64 --symbolica-n-cores 5
       --symbolica-output-chunk-size 128 --symbolica-output-chunk-strategy uniform --symbolica-stage-local-parameter-layout --symbolica-iterations 10 --symbolica-max-horner-scheme-variables 1000 --symbolica-max-common-pair-cache-entries 5000000
       --symbolica-max-common-pair-distance 1000 --lc-sector-ids 0 --reference-color-order 2,4,5,6,7,8,9,1,3 --symbolica-evaluator-backend jit --symbolica-jit-optimization-level 3
env PYTHONPATH=src dependencies/.venv/bin/python -m pyamplicol time-process --target-runtime 10 --batch-size 64 --json docs/.z_performance_outputs/manual/n7/jit_o3
```

9 Generic NLC Performance Matrix

Cell format: generation time above runtime per phase-space point; slots are AmpliCol, pyAmpliCol JIT O1, and pyAmpliCol C++ O3. AmpliCol uses raw generated-library colour probes. Conventions and gaps are summarized after the table.

ID	base process	n=1				n=2				n=3			
1	$d\bar{d} \to Z + (n-1)g$	1.913	(x0.069)	(x0.492)		2.052	(x0.076)	(x0.627)		2.003	(x0.104)	(x1.73)	
		11.02	(x0.017)	(x0.005)	(x0.017) (x0.005)	25.42	(x0.015)	(x0.008)	(x0.016) (x0.009)	88.93	(x0.019)	(x0.012)	(x0.021) (x0.016)
2	$u\bar{d} \to W^+ + (n-1)g$	1.874	(x0.069)	(x0.402)		2.018	(x0.077)	(x0.563)		1.974	(x0.088)	(x1.19)	
		10.06	(x0.016)	(x0.004)	(x0.017) (x0.004)	24.23	(x0.012)	(x0.006)	(x0.012) (x0.006)	70.51	(x0.014)	(x0.009)	(x0.017) (x0.012)
3	$d\bar{d} \to e^+e^- + (n-2)g$	N/A				2.05	(x0.078)	(x0.563)		2.001	(x0.078)	(x0.837)	
						16.91	(x0.015)	(x0.007)	(x0.016) (x0.007)	37.75	(x0.011)	(x0.006)	(x0.013) (x0.008)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	N/A				2.025	(x0.073)	(x0.52)		1.987	(x0.075)	(x0.716)	
						15.16	(x0.013)	(x0.005)	(x0.012) (x0.004)	31.1	(x0.009)	(x0.004)	(x0.009) (x0.004)
5	$d\bar{d} \to ZZ + (n-2)g$	N/A				2.084	(x0.076)	(x0.653)		2.059	(x0.092)	(x1.44)	
						41.38	(x0.011)	(x0.006)	(x0.012) (x0.007)	115	(x0.01)	(x0.006)	(x0.012) (x0.009)
6	$gg \to t\bar{t} + (n-2)g$	N/A				2.058	(x0.084)	(x0.885)		2.284	(x0.187)	(x7.21)	
						22.02	(x0.042)	(x0.022)	(x0.047) (x0.026)	182	(x0.051)	(x0.04)	(x0.058) (x0.046)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	N/A				2.079	(x0.076)	(x0.591)		2.226	(x0.104)	(x2.08)	
						18.18	(x0.027)	(x0.011)	(x0.028) (x0.012)	73.61	(x0.037)	(x0.023)	(x0.042) (x0.03)
8	$gg \to gg + (n-2)g$	N/A				2.217	(x0.09)	(x1.78)		3.041	(x0.389)	(x24.9)	
						53.49	(x0.031)	(x0.021)	(x0.04) (x0.031)	1.24e+03	(x0.026)	(x0.021)	(x0.03) (x0.027)
9	$d\bar{d} \to ZZZ + (n-3)g$	N/A				N/A				2.408	(x0.108)	(x2.46)	
										261	(x0.008)	(x0.006)	(x0.011) (x0.009)
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	N/A				N/A				N/A			
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	N/A				N/A				N/A			
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	N/A				N/A				N/A			
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	N/A				N/A				N/A			
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A				N/A				N/A			
summary: gen		1.87 |	1.91 |	1.89 |	1.89		2.02 |	2.22 |	2.07 |	2.06		1.97 |	3.04 |	2.22 |	2.06
		x0.069 |	x0.069 |	x0.069 |	x0.069	x0.069	x0.073 |	x0.09 |	x0.079 |	x0.076	x0.079	x0.075 |	x0.389 |	x0.136 |	x0.104	x0.149
summary: run		10.1 |	11 |	10.5 |	10.5		15.2 |	53.5 |	27.1 |	23.1		31.1 |	1.24e3 |	234 |	88.9	
		x0.016 |	x0.017 |	x0.017 |	x0.017	x0.017	x0.011 |	x0.042 |	x0.021 |	x0.015	x0.021	x0.008 |	x0.051 |	x0.02 |	x0.014	x0.024

Generic NLC Performance Matrix (continued)

ID	base process	n=4			n=5			n=6
1	$d\bar{d} \to Z + (n-1)g$	2.377 972	(x0.307) (x0.018) (x0.015)	(N/A) (N/A)	4.711 2.26e+04	(x1.93) (x0.023) (x0.022)	(N/A) (N/A)	N/A
2	$u\bar{d} \to W^+ + (n-1)g$	2.26 624	(x0.229) (x0.018) (x0.014)	(N/A) (N/A)	3.591 1.47e+04	(x1.67) (x0.022) (x0.02)	(N/A) (N/A)	N/A
3	$d\bar{d} \to e^+e^- + (n-2)g$	2.106 159	(x0.096) (x0.009) (x0.007)	(N/A) (N/A)	3.128 1.83e+03	(x0.225) (x0.008) (x0.006)	(N/A) (N/A)	N/A
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	2.021 101	(x0.082) (x0.008) (x0.005)	(N/A) (N/A)	2.652 966	(x0.155) (x0.007) (x0.006)	(N/A) (N/A)	N/A
5	$d\bar{d} \to ZZ + (n-2)g$	2.233 769	(x0.188) (x0.009) (x0.007)	(N/A) (N/A)	4.119 1.16e+04	(x0.63) (x0.008) (x0.007)	(N/A) (N/A)	N/A
6	$gg \to t\bar{t} + (n-2)g$	4.118 4.41e+03	(x1.17) (x0.071) (x0.063)	(N/A) (N/A)	26.71 1.38e+05	(x8.36) (x0.173) (x0.17)	(N/A) (N/A)	N/A
7	$d\bar{d} \to t\bar{t} + (n-2)g$	3.683 927	(x0.303) (x0.04) (x0.034)	(N/A) (N/A)	16.12 2.35e+04	(x1.20) (x0.072) (x0.068)	(N/A) (N/A)	N/A
8	$gg \to gg + (n-2)g$	12.99 4.24e+04	(x2.00) (x0.039) (x0.036)	(N/A) (N/A)	160 2.05e+06	(x11.1) (x0.073) (x0.072)	(N/A) (N/A)	N/A
9	$d\bar{d} \to ZZZ + (n-3)g$	2.98 1.25e+03	(x0.153) (x0.005) (x0.004)	(N/A) (N/A)	6.688 1.2e+04	(x0.275) (x0.003) (x0.003)	(N/A) (N/A)	N/A
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	3.867 102	(x0.058) (x0.016) (x0.012)	(N/A) (N/A)	4.664 358	(x0.063) (x0.008) (x0.007)	(N/A) (N/A)	N/A
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	2.676 189	(x0.119) (x0.021) (x0.017)	(N/A) (N/A)	4.206 1.81e+03	(x0.295) (x0.015) (x0.014)	(N/A) (N/A)	N/A
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	4.034 220	(x0.055) (x0.007) (x0.005)	(N/A) (N/A)	5.128 828	(x0.056) (x0.003) (x0.003)	(N/A) (N/A)	N/A
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	N/A N/A	0.27 (3.4)	N/A N/A	N/A N/A	1.385 (51.7)	N/A N/A	N/A
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A			N/A			N/A
summary: gen		2.02|	13| 3.78|	2.83		2.65|	160| 20.1|	4.69		N/A
		x0.055|	x2.00| x0.396|	x0.17|	x0.775	x0.056|	x11.1| x2.16|	x0.463|	x8.42	N/A
summary: run		101|	4.24e4| 4.35e3|	697		358|	2.05e6| 1.9e5|	1.18e4		N/A
		x0.005|	x0.071| x0.022|	x0.017|	x0.04	x0.003|	x0.173| x0.035|	x0.012|	x0.077	N/A

Each cell is raw-library AmpliCol, pyAmpliCol JIT O1, and pyAmpliCol C++ O3. Entries are generation seconds above runtime microseconds per point; pyAmpliCol runtime shows coloured wall and black pure-evaluator ratios, or absolute time without an AmpliCol reference. Green/orange/red means below one/below two/at least two. Summaries are `min|max|avg|med|sum`; `sum` compares paired totals. A **VALIDATION FAILED** marker means the same-point difference exceeds 10^{-8}.

Settings. `docs/result_matrix.py` (`-color-accuracy nlc`) writes this table and its JSON cache. AmpliCol uses the raw generated-library path `amplicol_generate -library=create-raw` followed by `amplicol_color_library_probe`, preserving NLC/full colour-order interferences. pyAmpliCol uses Rusticol double precision, batch size 64, output chunk size 128, Stage-local inputs, ten Horner iterations, default CPE rounds, expanded optimization limits, SymJIT O1, and `-target-runtime 10` are used. Only C++ O3 generation is subject to the 15-minute compile cap.

Fortran reference gaps. 2 completed pyAmpliCol cells have no direct Fortran colour reference because that path stops above two quark lines; they are shown as absolute timings.

C++ O3 gaps. 24 optional cells were not run and 0 exceeded the 15-minute compile budget; completed AmpliCol/JIT data remain shown.

10 Generic Full-Colour Performance Matrix

Cell format: generation time above runtime per phase-space point; slots are AmpliCol, pyAmpliCol JIT O1, and pyAmpliCol C++ O3. AmpliCol uses raw generated-library colour probes. Conventions and gaps are summarized after the table.

ID	base process	n=1			n=2			n=3		
1	$d\bar{d} \to Z + (n-1)g$	1.885	(x0.071)	(N/A)	2.061	(x0.076)	(N/A)	2.038	(x0.103)	(N/A)
		10.75	(x0.018) (x0.005)	(N/A)	26.31	(x0.015) (x0.007)	(N/A)	88.47	(x0.019) (x0.012)	(N/A)
2	$u\bar{d} \to W^+ + (n-1)g$	1.908	(x0.071)	(N/A)	2.023	(x0.075)	(N/A)	1.987	(x0.09)	(N/A)
		10.13	(x0.016) (x0.004)	(N/A)	23.57	(x0.013) (x0.006)	(N/A)	68.26	(x0.015) (x0.01)	(N/A)
3	$d\bar{d} \to e^+e^- + (n-2)g$	N/A			2.054	(x0.073)	(N/A)	2.017	(x0.078)	(N/A)
					16.08	(x0.016) (x0.007)	(N/A)	38.14	(x0.011) (x0.006)	(N/A)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	N/A			2.026	(x0.073)	(N/A)	1.976	(x0.074)	(N/A)
					15.18	(x0.013) (x0.005)	(N/A)	32.16	(x0.009) (x0.004)	(N/A)
5	$d\bar{d} \to ZZ + (n-2)g$	N/A			2.098	(x0.076)	(N/A)	2.058	(x0.092)	(N/A)
					39.52	(x0.012) (x0.006)	(N/A)	116	(x0.009) (x0.006)	(N/A)
6	$gg \to t\bar{t} + (n-2)g$	N/A			2.073	(x0.083)	(N/A)	2.275	(x0.187)	(N/A)
					22.03	(x0.042) (x0.022)	(N/A)	184	(x0.055) (x0.039)	(N/A)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	N/A			2.057	(x0.076)	(N/A)	2.222	(x0.104)	(N/A)
					18.26	(x0.027) (x0.011)	(N/A)	74.1	(x0.036) (x0.023)	(N/A)
8	$gg \to gg + (n-2)g$	N/A			2.161	(x0.092)	(N/A)	3.03	(x0.415)	(N/A)
					52.76	(x0.03) (x0.021)	(N/A)	1.26e+03	(x0.03) (x0.02)	(N/A)
9	$d\bar{d} \to ZZZ + (n-3)g$	N/A			N/A			2.415	(x0.109)	(N/A)
								261	(x0.009) (x0.006)	(N/A)
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	N/A			N/A			N/A		
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	N/A			N/A			N/A		
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	N/A			N/A			N/A		
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	N/A			N/A			N/A		
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A			N/A			N/A		
summary: gen		1.89 |	1.91 |	1.9 | 1.9		2.02 |	2.16 |	2.07 | 2.06		1.98 |	3.03 |	2.22 | 2.06
		x0.071 |	x0.071 |	x0.071 | x0.071 x0.071		x0.073 |	x0.092 |	x0.078 | x0.076 x0.078		x0.074 |	x0.415 |	x0.139 | x0.103 x0.153
summary: run		10.1 |	10.7 |	10.4 | 10.4		15.2 |	52.8 |	26.7 | 22.8		32.2 |	1.26e3 |	236 | 88.5
		x0.016 |	x0.018 |	x0.017 | x0.017 x0.017		x0.012 |	x0.042 |	x0.021 | x0.015 x0.022		x0.009 |	x0.055 |	x0.021 | x0.015 x0.027

Generic Full-Colour Performance Matrix (continued)

ID	base process	n=4			n=5			n=6
1	$d\bar{d} \to Z + (n-1)g$	2.454 975	(x0.304) (x0.02) (x0.015)	(N/A) (N/A)	4.865 2.33e+04	(x1.96) (x0.026) (x0.022)	(N/A) (N/A)	N/A
2	$u\bar{d} \to W^+ + (n-1)g$	2.271 630	(x0.235) (x0.018) (x0.014)	(N/A) (N/A)	3.687 1.52e+04	(x1.68) (x0.023) (x0.02)	(N/A) (N/A)	N/A
3	$d\bar{d} \to e^+e^- + (n-2)g$	2.124 160	(x0.097) (x0.009) (x0.007)	(N/A) (N/A)	3.188 1.92e+03	(x0.227) (x0.008) (x0.006)	(N/A) (N/A)	N/A
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	2.045 103	(x0.082) (x0.008) (x0.005)	(N/A) (N/A)	2.745 1.01e+03	(x0.157) (x0.007) (x0.006)	(N/A) (N/A)	N/A
5	$d\bar{d} \to ZZ + (n-2)g$	2.225 787	(x0.192) (x0.009) (x0.006)	(N/A) (N/A)	4.233 1.18e+04	(x0.631) (x0.008) (x0.007)	(N/A) (N/A)	N/A
6	$gg \to t\bar{t} + (n-2)g$	4.145 4.44e+03	(x1.17) (x0.078) (x0.062)	(N/A) (N/A)	27.28 1.46e+05	(x8.64) (x0.175) (x0.159)	(N/A) (N/A)	N/A
7	$d\bar{d} \to t\bar{t} + (n-2)g$	3.687 941	(x0.302) (x0.043) (x0.033)	(N/A) (N/A)	16.14 2.42e+04	(x1.21) (x0.071) (x0.063)	(N/A) (N/A)	N/A
8	$gg \to gg + (n-2)g$	13 4.35e+04	(x2.33) (x0.041) (x0.035)	(N/A) (N/A)	157 2.03e+06	(x13.2) (x0.082) (x0.075)	(N/A) (N/A)	N/A
9	$d\bar{d} \to ZZZ + (n-3)g$	3 1.24e+03	(x0.149) (x0.005) (x0.004)	(N/A) (N/A)	6.811 1.24e+04	(x0.286) (x0.003) (x0.003)	(N/A) (N/A)	N/A
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	3.992 103	(x0.056) (x0.016) (x0.012)	(N/A) (N/A)	4.768 366	(x0.063) (x0.008) (x0.007)	(N/A) (N/A)	N/A
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	2.668 188	(x0.12) (x0.022) (x0.016)	(N/A) (N/A)	4.311 1.87e+03	(x0.292) (x0.015) (x0.013)	(N/A) (N/A)	N/A
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	4.003 222	(x0.055) (x0.007) (x0.005)	(N/A) (N/A)	5.236 844	(x0.056) (x0.003) (x0.003)	(N/A) (N/A)	N/A
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	N/A N/A	0.269 (3.46)	N/A N/A	N/A N/A	1.413 (55.5)	N/A N/A	N/A
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A			N/A			N/A
summary: gen		2.04|	13|	3.8|	2.83		2.75|	157|	20|	4.82		N/A
		x0.055|	x2.33|	x0.424|	x0.17|	x0.866	x0.056|	x13.2|	x2.37|	x0.462|	x9.81	N/A
summary: run		103|	4.35e4|	4.44e3|	709		366|	2.03e6|	1.9e5|	1.21e4		N/A
		x0.005|	x0.078|	x0.023|	x0.017|	x0.042	x0.003|	x0.175|	x0.036|	x0.012|	x0.086	N/A

Each cell is raw-library AmpliCol, pyAmpliCol JIT O1, and pyAmpliCol C++ O3. Entries are generation seconds above runtime microseconds per point; pyAmpliCol runtime shows coloured wall and black pure-evaluator ratios, or absolute time without an AmpliCol reference. Green/orange/red means below one/below two/at least two. Summaries are `min|max|avg|med|sum`; `sum` compares paired totals. A **VALIDATION FAILED** marker means the same-point difference exceeds 10^{-8}.

Settings. `docs/result_matrix.py` (`-color-accuracy full`) writes this table and its JSON cache. AmpliCol uses the raw generated-library path `amplicol_generate -library=create-raw` followed by `amplicol_color_library_probe`, preserving NLC/full colour-order interferences. pyAmpliCol uses Rusticol double precision, batch size 64, output chunk size 128, Stage-local inputs, ten Horner iterations, default CPE rounds, expanded optimization limits, SymJIT O1, and `-target-runtime 10` are used. Only C++ O3 generation is subject to the 15-minute compile cap.

Fortran reference gaps. 2 completed pyAmpliCol cells have no direct Fortran colour reference because that path stops above two quark lines; they are shown as absolute timings.

C++ O3 gaps. 43 optional cells were not run and 0 exceeded the 15-minute compile budget; completed AmpliCol/JIT data remain shown.

Archived low-multiplicity validation fixture. The $n \leq 4$ LC/NLC/full-colour validation cells are mirrored in `tests/fixtures/result_matrix_references.json`. The integration test checks those cached same-point AmpliCol/pyAmpliCol values without a local Fortran run; `scripts/refresh_result_matrix_fixtures.py` refreshes the fixture from the matrix caches.

11 Why This Matches AmpliCol but Remains Python-Friendly

The Python layer owns graph construction, process metadata, deterministic phase-space steering, artifact generation, evaluator caching, and validation against AmpliCol. It does not perform the hot current recursion component by component in production. The two production execution routes are now:

Python staged route: source fill → bounded stage evaluators → amplitude/raw-sum evaluator,

Rusticol route: one PyO3 call → Rust source fill and staged sweep → amplitude/raw-sum evaluator.

Thus pyAmpliCol keeps AmpliCol's essential optimization - shared currents across helicity parents and a topological current-table sweep - while replacing generated Fortran subroutines by saved Spenso/Symbolica evaluator artifacts. The `rusticol` path removes the remaining Python orchestration cost by loading the same process directory and executing the sweep in Rust.

The old fully monolithic compiled-DAG route is now deprecated and kept only as reference code. Its alias-based design was useful for exploring whole-graph compilation, but the production target is the staged DAG: Python generation plus a Rust eager-DAG runtime over saved Symbolica stage evaluators. This keeps the runtime schedule close to AmpliCol's current-table library while leaving the graph compiler model-driven and process-generic.